

C++控制台贪吃蛇 完整学习笔记

项目基础信息

1. 项目归属: CPlusPlus-Basic-Learning 仓库 → 08_Game-Demo → snake.cpp
2. 运行环境: 仅 Windows 系统 (依赖 windows.h、conio.h 专属头文件, Linux/Mac 无法运行)
3. 前置基础: 大一 C++必修 (分支判断、循环、一维/二维数组、结构体、函数封装、随机数、控制台 API)
4. 项目难度: 大一期末课程设计级别, 无 STL 高级语法、无第三方图形库, 纯标准 C++/Windows 控制台 API
5. 核心功能: 方向键控制蛇移动、吃食物变长、撞墙/撞身体游戏结束、分数计分、难度调速

一、项目用到全部核心知识点

(一) 基础语法知识点

1. 分支结构: if-else 多分支判断、多条件逻辑判断 (判断撞墙、撞自身、吃到食物)
2. 循环结构: while 死循环做游戏主循环、for 嵌套循环绘制游戏地图、遍历蛇身体节点
3. 基础变量: 全局变量优势 (简化函数传参, 小游戏通用写法)、局部变量作用域、常量定义 (地图长宽)
4. 运算符: 逻辑运算符 &&||、赋值运算符、自增自减 (蛇身体坐标更新)

(二) 核心进阶知识点

1. 结构体 struct

- 定义蛇身体坐标结构体, 存储每一节蛇身 x/y 坐标
- 数组结构体: 用结构体数组存储整条蛇所有节点, 实现蛇身长动态增减
- 大一核心考点: 结构体初始化、结构体数组遍历、结构体成员调用

2. 二维数组

- 构建游戏地图棋盘, 划分空地、墙体、蛇身、食物四种地图状态
- 二维下标对应控制台横纵坐标, 实现控制台点位绘制

3. 随机数系统

- srand((unsigned)time(NULL)) 设置随机数种子
- rand() 生成随机坐标, 随机刷新食物位置
- 避坑: 必须引入 <ctime> 头文件, 防止食物固定生成

4. Windows 专属控制台 API (项目特色知识点)

1. _kbhit(): 无阻塞键盘监听, 不等待回车, 实时检测是否按下按键 (区别普通 cin 输入)
 2. getch(): 无回显读取按键, 读取上下左右方向键 ASCII 码
 3. SetConsoleCursorPosition(): 移动控制台光标, 实现局部刷新画面 (不用清屏闪屏)
 4. system("cls"): 全屏清屏; Sleep(ms): 控制游戏帧率, 调节移动速度
- ##### 5. 游戏逻辑算法
1. 蛇移动算法: 尾部赋值前移法 (核心算法): 蛇最后一节身体赋值上一节坐标, 头部按方

向移动，最简可理解算法

2. 方向互斥判定：禁止蛇直接反向掉头（向右不能直接向左，防止自杀）

3. 碰撞检测算法：边界坐标碰撞、自身节点坐标重合碰撞

6. 函数模块化封装

拆分功能函数：地图初始化、蛇初始化、食物生成、画面绘制、按键监听、移动逻辑、碰撞判定，贴合代码工程化思想，适配 GitHub 项目规范

二、代码模块拆分&对应知识点详解

模块 1：头文件区（区分标准 C++Windows 专属头文件）

1. 标准 C++头文件（跨平台通用）：

`：输入输出打印地图、分数

- <ctime>：配合 time 生成随机食物

- using namespace std;：大一简化写法，省去 std::前缀

2. Windows 专属头文件（项目限制关键）

- #include<windows.h>：光标移动、Sleep 延时函数

- #include<conio.h>：_kbhit、getch 键盘无阻塞监听

仓库备注知识点：跨平台兼容性缺陷，Linux 需替换 ncurses 库，本项目仅限 Windows

模块 2：宏定义全局常量

cpp

```
#define WIDTH 30 //地图宽
```

```
#define HEIGHT 20 //地图高
```

知识点：宏定义常量，修改数值即可一键更改地图大小，代码高可维护，GitHub 项目规范写法

模块 3：结构体设计（核心重难点）

cpp

```
//蛇身坐标结构体
```

```
struct Snake{
```

```
    int x;
```

```
    int y;
```

```
};
```

```
Snake snake[100]; //结构体数组，最长 100 节蛇身
```

知识点:

1. 自定义结构体封装坐标属性，贴合面向对象封装思想入门
2. 结构体数组：下标 0 为蛇头，下标越大越靠蛇尾，遍历从尾部更新坐标

模块 4：核心全局变量作用

- `int len`：蛇初始长度
- `int fx`：移动方向（上下左右用数字 1/2/3/4 标记）
- `int foodX, foodY`：食物坐标
- `int score`：游戏分数

知识点：小游戏全局变量简化函数传参，减少参数冗余，大一项目最优写法

模块 5：六大功能函数拆解

1. `InitGame()`：游戏初始化

知识点：结构体初始化赋值、随机数种子初始化、初始方向赋值

2. `DrawMap()`：绘制地图墙体、蛇、食物

知识点：二维循环打印控制台字符、光标定位绘制、局部画面刷新

3. `KeyControl()`：按键监听

重难点：`_kbhit()` 实时监听、方向键双字节 ASCII 码判定、反向移动拦截逻辑

4. `SnakeMove()`：蛇移动核心逻辑

必考算法：从尾到头赋值前移

示例逻辑：`snake[i]=snake[i-1]`，所有身体跟随前一节移动，仅蛇头更改坐标

5. `FoodCreate()`：食物重生

知识点：随机数取模限定地图范围、判断食物不能生成在蛇身上

6. `IsDead()`：死亡判定

知识点：边界坐标判断、循环遍历结构体数组判断蛇头触碰身体

模块 6：main 主函数

知识点：`while(1)` 游戏死循环、`Sleep` 帧率调速、游戏状态流转（运行→死亡结束）

三、高频易错点&踩坑总结

1. 方向键 bug：方向键属于扩展按键，`getch` 会读取两个值，必须做二次读取判断，直接判断会按键失效
2. 反向自杀 bug：未做方向拦截，向右移动直接按左键，蛇头直接撞身体死亡，必须增加 `fx` 互斥判断
3. 闪屏 bug：全程用 `system("cls")` 全屏清屏，画面剧烈闪烁，优化方案：光标定位局部覆盖绘制，不清屏
4. 随机食物重复 bug：未写食物避蛇判定，食物刷新在蛇身上，直接自动加分
5. 全局变量误用：局部定义蛇结构体，无法跨函数修改蛇坐标，新手极易报错
6. `Sleep` 延时误区：延时越小速度越快，可绑定分数动态提速，实现难度递增

四、项目能力提升

1. 代码工程化：拆分单一功能函数，可读性高，可直接上传仓库
2. 逻辑思维：掌握回合制游戏主循环+状态判定通用开发模型，可复刻俄罗斯方块、贪吃蛇、打砖块
3. 算法迁移：尾部前移算法可复用至所有轨迹移动类控制台游戏
4. 拓展改造方向（自主进阶）：
 - 增加难度档位选择
 - 增加穿墙模式
 - 存储最高分 txt 文件读写（拓展文件读写知识点）

五、仓库配套 README 撰写要点

贪吃蛇 Snake.cpp

1. 游戏玩法

- 上下左右方向键控制蛇移动
- 吃到★食物加分、蛇身变长
- 撞边界、撞到自身游戏结束
- 分数越高，移动速度越快

2. 涉及知识点

结构体、结构体数组、二维循环、Windows 控制台 API、无阻塞键盘监听、随机数、碰撞算法、模块化函数、游戏主循环

3. 运行须知

仅限 Windows 系统运行，依赖 windows.h、conio.h 头文件，Visual Studio/Dev-C++均可编译运行

4. 拓展优化

支持自定义地图大小、动态调速、最高分存档